

COMPANION TRAINING CURRICULUM

Zero to Mastery Enterprise Agentic AI Security Architect

A 10-Week, Instructor-Ready Curriculum for IT and Software Professionals With No Security Background

Document Code: EASA-ZTM

Version: 1.0

Date: June 2026

Scope: Companion to EASA-01 through EASA-06

Enterprise Agentic AI Security Architect (2026–2030) Master Research Program

Table of Contents

How to Use This Curriculum

This is the instructional companion to the six-volume Enterprise Agentic AI Security Architect (2026–2030) Master Research Program (document codes EASA-01 through EASA-06). Those six volumes are the reference architecture — dense, comprehensive, written for someone who already knows what question they're trying to answer. This document is the path that gets a learner to the point where they can read those volumes and get full value from them, then keeps going until that learner can produce the kind of work those volumes model.

The starting assumption is explicit: learners enter this curriculum with general IT or software experience — they can read an architecture diagram, they have used cloud infrastructure, they may have written code — but they do not have a security background. No prior knowledge of threat modeling, identity and access management, or AI/ML systems is assumed. Stage 0 exists specifically to build that floor before any agent-specific material is introduced, because the single most common failure mode in technical training is skipping the fundamentals to get to the exciting part faster, which produces learners who can repeat vocabulary without being able to reason from it.

Program Structure

Ten stages, each mapped to roughly one week of full-time study (a working week of approximately 35–40 hours), organized into four phases:

Phase	Stages	Focus	Maps to Reference Volume
Phase I — Foundations	0–1	Security fundamentals and core agentic architecture	EASA-01
Phase II — The Core Stack	2–4	Identity & NHI, MCP security, A2A security	EASA-02
Phase III — Governance & Operations	5–7	Governance/compliance, AI SOC & observability, red teaming & reliability	EASA-03, EASA-04
Phase IV — Economics & Mastery	8–9	Economic security (FinOps/commerce/PQC); capstone & operating model	EASA-05, EASA-06

Every Stage Follows the Same Structure

- **Objectives** — a short, specific list of what you will be able to do, not just what you will have read, by the end of the stage.
- **Prerequisite check** — a one-line gate confirming you have what the stage assumes before you start; if you can't honestly check this box, go back.
- **Core content** — the conceptual material, written to teach rather than to reference — expect more explanation and more "why," and less density, than the EASA-0X reference volumes.
- **Hands-on lab** — a concrete, scoped exercise that produces a real artifact (a policy file, a threat model, a registry schema, a working gateway config). This is where the learning actually consolidates; do not skip the labs.

- **Knowledge check** — questions designed to expose gaps, not to be answered from memory. If you can't answer one without going back to the material, that's useful information, not a failure.
- **Stage capstone** — a larger deliverable, usually building on the lab, scoped to take a meaningful slice of the week and reviewable against a stated rubric.

Pacing for an Instructor or Cohort

Day	Suggested Activity
Mon–Tue	Core content; instructor-led walkthrough if running as a cohort; independent reading otherwise
Wed–Thu	Hands-on lab; pair or small-group work recommended where the lab involves a multi-component system (e.g., a gateway)
Fri	Knowledge check, stage capstone work session, and capstone submission

For instructors running this as a cohort

Every stage capstone is designed to be reviewable in roughly 20–30 minutes per learner using the stated rubric. Build a standing Friday-afternoon review slot into the schedule. For Stages 2 through 4 (the identity/MCP/A2A core), consider running labs in pairs — these stages have the steepest learning curve in the program, and the production-incident research cited throughout EASA-02 makes a strong case that most real-world failures in this space come from working alone under time pressure with no second reviewer, which is exactly the dynamic pairing is meant to counteract.

What "Mastery" Means at the End of This Program

Mastery here is defined operationally, not by a credential. A learner who completes this curriculum should be able to walk into an organization deploying its first significant agent population and do four things without supervision:

1. Threat-model a proposed agentic system using MAESTRO, identify which OWASP ASI categories apply, and produce a prioritized control list that a platform engineering team could actually implement.
2. Design and defend an identity, MCP, and A2A architecture for that system — including the specific tradeoffs (isolation tier, autonomy level, gateway placement) and being able to explain why each choice was made, not just what the choice was.
3. Map that system's risk profile against the governance frameworks that actually apply to it (ISO 42001, relevant regulation, internal policy) and produce the crosswalk evidence an auditor would need.
4. Operate the system after launch — read its telemetry, recognize the difference between a reliability problem and a security incident, run a red-team exercise against it, and know when to pull the kill switch.

The capstone in Stage 9 is built to require exactly these four things, applied to a single realistic scenario, because that is the actual job.

STAGE 0 • Week 1

Security Fundamentals for Builders

If you have never worked in security, this stage is not optional, and it is not filler. Every subsequent stage assumes you already think in terms of threat actors, trust boundaries, and the difference between authentication and authorization. Most of the costly mistakes documented in the reference volumes — over-permissioned agents, conflated identity and trust, security bolted on after deployment — trace back to someone skipping exactly this kind of foundation.

BY THE END OF THIS STAGE, YOU WILL BE ABLE TO

- Explain the difference between authentication, authorization, and accountability, with an example of a system that gets each one wrong
- Identify a trust boundary in a system diagram and explain what crossing it should require
- Apply the CIA triad (confidentiality, integrity, availability) and explain why it is insufficient on its own for systems that act autonomously
- Read and produce a basic STRIDE threat enumeration for a simple system
- Explain defense in depth, zero trust, and assume breach in your own words, with a concrete counter-example for each

Prerequisite check: None — this is the entry point. If you already hold a security certification (Security+, CISSP, etc.) or have 2+ years in a security role, you can likely skim this stage and move to its knowledge check directly.

0.1 Why Security Thinking Is Different From Engineering Thinking

Most engineering disciplines optimize for the system working correctly under expected conditions. Security is the discipline of reasoning about the system under adversarial conditions — when someone is deliberately trying to make it behave incorrectly, and is willing to invest real effort to find the one path you didn't think to defend. This is a different mental motion than debugging: debugging asks "why isn't this working as intended," security asks "what is the worst thing someone could make this do, and how would I know if they were trying."

The practical consequence: a security review is never "does this work" — it's "what happens at the edges, under hostile input, when an assumption I made turns out to be false." You will be asked to hold this posture constantly once agentic systems enter the picture, because an agent is, among other things, a system that can be talked into attacking itself.

0.2 Authentication, Authorization, and Accountability

These three terms are used loosely in casual conversation and precisely in security work, and conflating them is the single most common root cause of access-control failures.

Term	Question It Answers	Common Failure When Conflated With the Others
Authentication	Who (or what) is this?	A system trusts that proving identity is the same as proving the action should be allowed — "I know who you are"

Term	Question It Answers	Common Failure When Conflated With the Others
		silently becomes "so you can do anything"
Authorization	Is this identity allowed to do this specific thing?	Authorization decisions are made once at login and never re-checked per action, so a legitimately authenticated session can be used for actions never actually approved
Accountability	Can we prove, after the fact, who did what?	Logging is treated as a nice-to-have rather than a structural requirement, so an incident can't be reconstructed even when the first two controls worked correctly

Hold onto this table. Every identity architecture you will design from Stage 2 onward is, at its core, an attempt to get these three things right simultaneously for a population of non-human actors that change faster than any human identity ever did.

0.3 Trust Boundaries

A trust boundary is any point in a system where data or control crosses from one zone of trust into another — from the internet into your network, from an unauthenticated user into an authenticated session, from a low-privilege process into a high-privilege one. Every trust boundary is a place where you must assume the thing on the other side might be lying, malformed, or malicious, and where you therefore need an explicit control (validation, authentication, authorization) rather than an implicit assumption of good behavior.

The mistake this concept exists to prevent

A system processes user input, passes it to an internal service, which passes it to a database query — and validation only happens at the very first step, because "it's already been checked." The internal service crossed a trust boundary (it's now operating on data that originated outside the system) without re-validating. This exact pattern, with "user input" replaced by "content an AI agent reads as part of its context," is the root mechanism behind prompt injection — which you'll study in depth starting in Stage 1.

0.4 The CIA Triad — and Why It's Not Enough Here

Confidentiality (only authorized parties can read it), Integrity (only authorized parties can change it, and you can detect if it was changed anyway), and Availability (it works when it's supposed to) is the classical foundation of information security and remains necessary. It is not sufficient for agentic systems, because none of the three categories captures the failure mode of a system that is confidential, has perfect integrity, is fully available — and is doing the wrong thing entirely because it was convinced to, of its own accord, by something it read. You'll see this gap named explicitly later as the motivation for "cognitive security" (EASA-04). For now, just notice that it's a gap: hold the question "what does the CIA triad not cover" in mind as you go.

0.5 STRIDE — Your First Threat Modeling Tool

STRIDE is a checklist for systematically asking "what could go wrong here" instead of relying on whatever threats happen to occur to you. Each letter is a category of thing an attacker might do to a component or a data flow in your system.

Letter	Category	Plain-Language Question
S	Spoofing	Can someone pretend to be someone or something they're not?
T	Tampering	Can someone change data or code without authorization?
R	Repudiation	Can someone deny having done something, because there's no proof they did?
I	Information Disclosure	Can someone see data they shouldn't be able to see?
D	Denial of Service	Can someone make the system unavailable to legitimate users?
E	Elevation of Privilege	Can someone gain more access than they were granted?

You will use STRIDE constantly through this program — it's the entry-level tool that the more advanced frameworks (MAESTRO, in Stage 1) extend rather than replace.

0.6 Three Governing Principles You'll See Everywhere From Here On

- **Defense in Depth** — no single control should be load-bearing. If one layer fails, another should still catch the problem. A locked door and a security camera and an alarm system are defense in depth; a locked door alone is not.
- **Zero Trust** — no actor, system, or network location is trusted by default just because of where it sits or what it successfully did a moment ago; every request is evaluated on its own merits at the point of use.
- **Assume Breach** — design as though something in your system is already compromised right now, and ask what limits the damage, rather than designing only to prevent compromise in the first place.

HANDS-ON LAB Threat-Model a Simple Web Application

You're given a basic three-tier system: a web frontend, an API backend, and a database, where the API backend also calls one external third-party service (a weather API) to enrich responses.

Tasks:

1. Draw the system as a simple diagram with three components and the third-party service, marking every place data crosses between them.
2. Identify every trust boundary in the diagram (there are at least four).
3. For each trust boundary, run through STRIDE and identify at least one plausible threat per applicable category — not every category will apply at every boundary, and explaining why a category doesn't apply is as valuable as finding one that does.
4. For each identified threat, write one sentence describing a control that would address it, and name which of authentication, authorization, or accountability (or which CIA triad property) it primarily protects.

Deliverable: A one-page threat model: the diagram, the labeled trust boundaries, and a table of (boundary, STRIDE category, threat, control) rows — minimum 8 rows.

KNOWLEDGE CHECK Stage 0

1. A system authenticates a user via username and password, then allows that authenticated session to call any API endpoint with no further checks. Which of the three concepts from section 0.2 has been skipped, and what's the risk?
2. Give an example of a trust boundary in a system you've personally worked on (any system, not necessarily security-related) that you hadn't previously thought of as a trust boundary.
3. Describe a system that is fully confidential, has perfect integrity, and is 100% available, but is still insecure. What does this tell you about the limits of the CIA triad?
4. Pick any household object that has a lock (a door, a safe, a bike lock) and describe what "defense in depth" would look like added to it, without just adding a second lock of the same kind.

STAGE 1 • Week 2

Agentic AI Architecture & Threat Modeling

With the Stage 0 vocabulary in place, this stage introduces what's actually new and different about agentic AI systems compared to the conventional software you threat-modeled in Stage 0 — and gives you the frameworks (MAESTRO specifically) built for this new category of system. This stage corresponds to EASA-01.

BY THE END OF THIS STAGE, YOU WILL BE ABLE TO

- Explain what makes an AI agent different from a conventional application or a simple chatbot, in terms of attack surface
- Walk through the canonical agent lifecycle (user → planner → agent → tool → memory → action) and name a distinct threat at each stage
- Apply the CSA MAESTRO seven-layer framework to a simple agentic system and identify at least one cross-layer threat
- Compare runtime isolation options (Firecracker, Kata, gVisor, WASM) and justify a choice for a given trust-boundary scenario
- Explain the 'credential custody lives outside the agent' design pattern and why it matters

***Prerequisite check:** Comfortable with Stage 0 vocabulary (trust boundaries, STRIDE, defense in depth) and basic familiarity with how an LLM-based application is built (a system prompt, a model call, a response) — you do not need ML/AI engineering experience, just conceptual familiarity with using an LLM API.*

1.1 What Actually Changes When a System Becomes "Agentic"

A conventional application, even one that calls an LLM, has a predictable, enumerable set of code paths — you can read the source and know every possible action the system can take. An agentic system is different in a structural way: the sequence of actions it takes is decided at runtime, by the model's own reasoning, in response to whatever it currently believes about the situation — including beliefs that can be shaped by content the system processes along the way. This is the single idea underneath almost everything in this program: an agent's behavior is not fully determined by its code, the way a conventional program's behavior is. It's co-determined by its code and by whatever it reads.

Why this matters more than it sounds like it should

In conventional security, you defend the code. In agentic security, you have to also defend the content — every document, email, search result, tool response, and message from another agent that your system will read becomes part of its effective instruction set, whether you intended that or not. This is the root cause of nearly every major risk category you'll study from here forward (goal hijacking, prompt injection, tool poisoning, memory poisoning) — they are all variations on "untrusted content was treated with the authority of a trusted instruction."

1.2 The Agent Lifecycle and Where Things Go Wrong

Every agentic interaction, regardless of framework, follows the same five-stage shape. Learning to see this shape in any system you encounter — including ones with unfamiliar names and unfamiliar frameworks — is one of the most transferable skills in this program.

Stage	What Happens	Representative Threat
User → Planner	A request or trigger initiates the interaction; the system interprets intent	Prompt injection — the "intent" the planner interprets isn't fully under the legitimate user's control
Planner → Agent	A plan is formed (explicitly or implicitly) for how to satisfy the request	Goal hijacking — the plan pursues an objective the legitimate user never intended
Agent → Tool	The agent calls an external tool, API, or system to gather information or take action	Tool misuse — a legitimate tool is invoked with malicious or unintended parameters
Tool → Memory	Results are stored, persisted, or used to update the agent's ongoing context	Memory poisoning — false or malicious information becomes part of what the agent "remembers" and trusts later
Memory/Tool → Action	The agent takes a real-world action based on everything above	Irreversible unauthorized action — the action lands before anyone notices something went wrong upstream

Notice this is the same five-stage decomposition you'll see formalized with specific controls in EASA-01, Domain 4. Internalizing this shape now means that when you encounter a genuinely new agent framework you've never seen before, you'll be able to map it onto this lifecycle in minutes rather than needing to learn an entirely new mental model.

1.3 MAESTRO — Threat Modeling Built for Agents

STRIDE (Stage 0) asks good questions but has no native way to ask "what if the system was successfully convinced, through entirely legitimate-looking input, to do something harmful?" MAESTRO (Multi-Agent Environment, Security, Threat, Risk, and Outcome) was built specifically to close that gap, by decomposing an agentic system into seven layers and requiring you to look for threats both within each layer and across layers.

Layer	Plain-Language Description
L1 — Foundation Models	The underlying LLM(s) doing the reasoning — the "brain"
L2 — Data Operations	Where data comes from, how it's labeled, stored, and fed in (including RAG)
L3 — Agent Frameworks	The orchestration logic deciding what the agent does next
L4 — Deployment & Infrastructure	Where and how the agent actually runs (containers, cloud, network)
L5 — Evaluation & Observability	How you test and watch the system
L6 — Security & Compliance	Cross-cutting controls that should apply at every layer

Layer	Plain-Language Description
L7 — Agent Ecosystem	How this agent interacts with other agents, marketplaces, and external parties

The exercise that makes MAESTRO valuable, and the one you'll practice in this stage's lab, is not listing threats at each layer in isolation — it's finding the threat that only exists because of how two layers interact. A poisoned document at L2 (Data Operations) becomes dangerous specifically because L3 (the Agent Framework) treats retrieved content as trustworthy context. Neither layer is "broken" on its own; the danger is in the seam between them.

1.4 Runtime Isolation — Containing the Blast Radius

If an agent's behavior can be shaped by what it reads, the next best line of defense is limiting what it can physically reach if something does go wrong. This is what runtime isolation is for — and the right choice depends entirely on how much you trust the boundary the agent operates inside.

Technology	Isolation Strength	When You'd Choose It
Firecracker microVMs	Highest — hardware-virtualized	An agent that executes arbitrary model-generated code against anything production-adjacent
Kata Containers	High — VM-grade, container-compatible	Need VM isolation without rearchitecting an existing container-based system
gVisor	Medium-high — syscall interception	General-purpose sandboxing where full VM isolation isn't necessary
WASM/WASI	Medium — capability-based sandboxing	Tool/plugin execution where near-zero startup latency matters
Hardened standard containers	Lowest of these options	Low-trust-boundary agents (e.g., read-only, no code execution) paired with strict network controls

The pattern worth internalizing: classify agents by what trust boundary they cross (does it run generated code? process untrusted external content? reach sensitive internal systems?) and assign isolation strength to match — not one isolation technology applied uniformly regardless of risk.

1.5 Credential Custody Lives Outside the Agent

This is the single most important architectural pattern in this stage, and it will recur throughout the program: the agent itself should never hold standing, reusable credentials to anything. Instead, a separate component (a "tool broker") holds actual credentials, receives a policy-approved request from the agent, executes the call, and returns only the result — never the credential — to the agent. If the agent is compromised or manipulated, there is nothing valuable for it to exfiltrate, because it never had anything to exfiltrate in the first place.

HANDS-ON LAB MAESTRO Threat Model for a Customer Support Agent

A company deploys a single-purpose AI agent that reads incoming support tickets, has read access to a customer CRM, has read access to a product knowledge base, and can draft (but not send, pending human approval) email replies.

Tasks:

1. Map this system onto the five-stage agent lifecycle from section 1.2, identifying what plays the role of each stage.
2. Using the MAESTRO seven layers, identify at least one threat per layer that is plausible for this specific system (not every layer will be equally rich — L7 may be thin for a single-agent system, and that's fine to note explicitly).
3. Identify at least one cross-layer threat — a threat that only exists because of how two specific layers interact (e.g., L2 and L3, or L3 and L4).
4. Assign a runtime isolation tier to this agent using the section 1.4 framework, and justify your choice in two to three sentences.
5. Apply the credential-custody pattern from section 1.5: sketch (in words or a simple diagram) how this agent's CRM and knowledge-base access should be brokered rather than held directly.

Deliverable: A structured MAESTRO threat model document: lifecycle mapping, layer-by-layer threats (minimum 7), at least one explicit cross-layer threat with justification, an isolation-tier decision, and the credential-custody sketch.

Reference: EASA-01, Domains 1.5 and 4–7

KNOWLEDGE CHECK Stage 1

1. In your own words, explain why an agentic system's attack surface includes 'content it reads' in a way that a conventional application's attack surface does not.
2. Name the MAESTRO layer where a compromised third-party plugin would most directly threaten a system, and explain why a STRIDE-only threat model might miss the way that plugin compromise interacts with the agent's reasoning.
3. An agent needs to read files from cloud storage and occasionally execute a small data-transformation script the model generates on the fly. Which isolation tier would you assign, and what would change your answer if the script execution were removed entirely?
4. Explain, to someone unfamiliar with this program, why holding credentials inside the agent itself is a worse design than brokering them — without using the word 'broker' in your explanation.

STAGE CAPSTONE Phase I Capstone — Secure Architecture Sketch

Choose a simple agentic use case of your own design (something you might plausibly build — a meeting-notes agent, a code-review agent, an internal IT-helpdesk agent — anything with at least two tool integrations). Produce a one-to-two-page architecture document.

Requirements:

- A system diagram showing the agent, its tools, its memory/context sources, and all trust boundaries.
- A MAESTRO-based threat model covering at least five of the seven layers, with at least one cross-layer threat identified.
- A stated isolation tier with justification.

- A credential-custody design (how tool access is brokered, not held directly by the agent).
- A short paragraph (4–6 sentences) describing what you would need to learn in Phase II to actually secure this system's identity, tool access, and any agent-to-agent communication it might eventually need.

Graded against: Completeness of the MAESTRO pass (all required layers addressed, cross-layer threat present and justified), correctness of the isolation-tier reasoning, and whether the credential-custody design genuinely keeps standing credentials out of the agent rather than just relocating the problem.

Phase II — The Core Stack

Stages 2 through 4 are the heart of this program, and deliberately mirror the deep-dive priority of EASA-02. If you only have time to master one phase deeply, this is the one. Identity, MCP, and A2A are presented as three separate stages, but by the end of Stage 4 you should see them as one continuous trust chain — identity establishes who's acting, MCP governs what tools they can use, A2A governs how they trust each other. Treat the three stages as a single connected unit even though they're paced across three weeks.

STAGE 2 • Week 3

Identity & Non-Human Identity (NHI)

This is, by a wide margin, the highest-leverage stage in the entire curriculum. The closing guidance of EASA-02 is blunt about this: if an organization can only fund one part of this whole program, fund the identity substrate. Everything from here forward — MCP gateways, A2A trust, governance reporting — gets dramatically easier once every agent carries a verifiable, ephemeral, centrally governed identity, and dramatically harder if it doesn't.

BY THE END OF THIS STAGE, YOU WILL BE ABLE TO

- Explain why static API keys and shared service accounts are inadequate for agent populations, using the Stage 0 authentication/authorization framing
- Describe how SPIFFE/SPIRE issues workload identity, including what attestation means and what an SVID actually proves
- Explain the specific limitation of SPIFFE (it authenticates the workload, not the intent) and what has to be layered on top of it
- Design a compound identity model that distinguishes 'which agent' from 'on whose authority'
- Compare service accounts, OAuth On-Behalf-Of, RFC 8693 token exchange, and impersonation as delegation patterns, and recommend one for a given scenario

Prerequisite check: Comfortable with Stage 0's authentication/authorization/accountability distinction and Stage 1's credential-custody pattern — this stage answers the question Stage 1 deliberately left open.

2.1 The Problem in One Sentence

Most organizations deploying agents today have solved "can this agent prove it's the agent it claims to be" using static API keys or shared service accounts — which works, in the narrow sense that it authenticates something, but creates standing, long-lived, broadly-scoped credentials that sit in configuration files and environment variables, exactly the kind of asset an attacker (or a manipulated agent, per Stage 1) wants to find. The harder, mostly unsolved problem at most organizations is authorization: not "who is this" but "what should this specific agent be allowed to do, right now, for this specific task" — and the gap between those two questions is where almost every real agent-identity incident lives.

2.2 Why Agents Need a Different Identity Model Than Humans or Servers

Human identity (you log in once a day, your identity rarely changes) and traditional server identity (a server exists for months, has a stable network location) both assume relative stability. Agents share none of that stability: they are often ephemeral (spun up for a single task and destroyed), they proliferate rapidly (one engineer might "own" dozens of agents), and a single agent's blast radius can span many internal systems in one session. An identity system designed for humans-who-log-in-once-a-day or servers-that-exist-for-months will always be playing catch-up against a population that can change shape in minutes.

2.3 SPIFFE/SPIRE — Workload Identity for Agents

SPIFFE (Secure Production Identity Framework For Everyone) is an open standard for issuing cryptographic identity to workloads — processes, containers, and (increasingly) agents — rather than to humans. SPIRE is the reference implementation. Here's the mechanism in plain terms:

1. An agent process starts up inside some piece of infrastructure (a container, a VM, a Kubernetes pod).
2. A local SPIRE agent observes properties of that process — which Kubernetes service account it's running as, what container image it was built from, what cloud instance metadata applies — and checks those properties against rules ("selectors") registered in advance.
3. If the properties match a registered selector, SPIRE issues a short-lived cryptographic credential called an SVID (SPIFFE Verifiable Identity Document) — typically a short-lived X.509 certificate.
4. Two SPIFFE-identified processes can then establish mutual TLS directly with each other, using their SVIDs, without any shared secret or manual enrollment step.

What this actually buys you

An agent spun up automatically a few seconds ago, with no human ever having manually registered it, can prove cryptographically which legitimate process it is — and that proof expires quickly and is re-issued continuously, rather than being a static secret someone has to remember to rotate. This is the direct structural fix for the static-API-key problem in section 2.1.

The limitation you must hold onto

An SVID proves "this is the legitimate agent process running this specific container image." It proves nothing about what that process is about to do. SPIFFE authenticates the workload, not the intent — this is exactly the authentication-versus-authorization distinction from Stage 0, section 0.2, reappearing in its most consequential form yet. A perfectly authenticated agent, with a perfectly valid SVID, can still be the agent doing something it shouldn't, because identity and authorization are different questions answered by different layers.

2.4 Compound Identity — The Hardest Open Problem

Most real agent actions happen on behalf of someone or something else: a human user, a business process, another agent that delegated a task. The authorization decision has to account for both "which agent is this" (answered by SPIFFE) and "on whose authority is it acting right now" (not answered by SPIFFE at all). An agent with a perfectly valid SVID acting on behalf of a now-suspended user account should be blocked — but a workload-identity check alone has no way to know that, because it was never designed to carry that second piece of context.

The architectural answer is to carry both pieces of information together, as a structured claim — commonly an "actor" claim inside a JWT, following the pattern defined in RFC 8693 (OAuth 2.0 Token Exchange) — so every downstream system can see, in one place, both which agent is acting and on whose authority, in a form that can't be forged or silently dropped along the way.

2.5 Delegation Patterns Compared

Pattern	How It Works	When It's the Right Choice / Wrong Choice
Service Accounts	A static, shared non-human credential	Acceptable only for the lowest-risk, most narrowly scoped automation; wrong choice for anything with meaningful blast radius — this is the pattern section 2.1 is warning you about
OAuth On-Behalf-Of	Carries forward a human's authorization through a service chain	Good for relatively shallow, predictable delegation (one or two hops); strains under deep multi-agent delegation chains
RFC 8693 Token Exchange	Standardized mechanism for exchanging one token for another, explicitly supporting delegation and impersonation claims	The recommended default for agent-acts-for-user flows in 2026; still maturing in agent-platform adoption
Federated Identity	Cross-domain trust without a shared credential store	Useful for cross-organization trust, but federation agreements were built for human SSO cadence, not agents discovering new counterparties at runtime
Impersonation	The agent simply assumes the user's full identity	Avoid as a default pattern — it destroys the audit distinction between "the user did this" and "the agent did this for the user," which is exactly the accountability property from Stage 0, section 0.2

HANDS-ON LAB Design a Compound Identity Token

Continuing the customer support agent from Stage 1's lab: the agent now needs to draft and, after human approval, send email replies on behalf of the support team, sometimes triggered automatically by a ticket and sometimes explicitly invoked by a specific human agent on the support team.

Tasks:

1. Sketch (as a simple JSON structure, no need for exact JWT syntax) a token that captures both the agent's own workload identity and the human/process it's currently acting for, using the actor-claim pattern from section 2.4.
2. Identify three different scenarios in which this token's claims should change shape (e.g., autonomous ticket-triggered action vs. explicit human invocation vs. a suspended human account) and show what the token looks like in each case.
3. Write the authorization rule (in plain English, structured as an if/then) that a downstream email-sending system should apply using this token — and explicitly state what it should do if the 'acting for' principal is suspended, even though the agent's own SVID is perfectly valid.
4. Identify one delegation pattern from the section 2.5 table that would be the wrong choice for this scenario, and explain why in two sentences.

Deliverable: A short design document: the compound-identity token sketch (3 variants), the downstream authorization rule, and the delegation-pattern justification.

Reference: EASA-02, Domain 2

KNOWLEDGE CHECK Stage 2

1. Explain, without using the term 'SPIFFE,' what workload attestation actually checks, and why that's different from just trusting whatever credential shows up.
2. A teammate says 'we don't need authorization logic, our agents all have valid SPIFFE SVIDs, so we know exactly which agent is doing what.' What's wrong with this statement?
3. Why is impersonation (the agent simply taking on the user's full identity) a worse design than a compound identity token, even though both let the agent 'act as' the user?
4. Describe a realistic scenario in your own work (or a hypothetical one) where RFC 8693 token exchange would be a better fit than plain OAuth On-Behalf-Of.

STAGE 3 • Week 4

Model Context Protocol (MCP) Security

MCP is how an agent talks to tools — and it is, as of 2026, the single most actively exploited layer in the agentic stack, with documented vulnerability research showing the majority of public MCP servers carry at least one serious flaw class. This stage takes the credential-custody and identity foundations from Stages 1–2 and applies them to the specific, currently-live threat landscape around MCP.

BY THE END OF THIS STAGE, YOU WILL BE ABLE TO

- Explain MCP's protocol basics (JSON-RPC, SSE vs. STDIO transport) and where each transport's biggest risk lives
- Identify tool poisoning, tool spoofing/rug pulls, dynamic tool escalation, tenant escape, and schema manipulation, and distinguish each from the others with an example
- Explain why discovery-time validation alone is insufficient and design a two-gate (discovery + invocation) validation approach
- Build a working MCP gateway policy that enforces allowlisting, schema validation, and identity-based authorization
- Evaluate a real or hypothetical MCP server against a security checklist and produce a pass/fail determination with justification

Prerequisite check: Comfortable with the credential-custody pattern (Stage 1.5) and compound identity (Stage 2.4) — MCP gateway design in this stage assumes you can use both.

3.1 MCP in One Paragraph

The Model Context Protocol standardizes how an AI agent discovers and calls external tools — a database query tool, a file-search tool, a calendar tool, anything. It's built on JSON-RPC 2.0 messaging over one of two transports: STDIO (the MCP server runs as a local subprocess, inheriting the privilege of whatever is running it — common for IDE-integrated coding assistants) or SSE/HTTP (a remote, networked server shared across an organization). The protocol's specification leaves a lot of authentication and session-handling behavior up to whoever implements a given server, which is the root cause of most of what follows in this stage.

3.2 Five Attack Patterns You Need to Recognize on Sight

- **Tool Poisoning** — a tool's description — text the agent reads as part of its reasoning context, that a human reviewer typically never sees — contains hidden instructions. A tool labeled "fetch data from S3" can also instruct the agent, invisibly, to exfiltrate results elsewhere. This is the MCP-specific instance of the "content becomes instructions" problem from Stage 1.1.
- **Tool Spoofing / Rug Pulls** — a malicious server registers a tool with a name nearly identical to a legitimate one (tool name collision), or a previously safe, reviewed tool is silently modified after installation to behave maliciously — meaning a point-in-time security review gives no durable guarantee.
- **Dynamic Tool Escalation** — an agent is granted a broad tool for a low-risk initial task, and that same standing grant is reused later for a higher-risk action nobody actually approved — privilege creep, MCP-flavored.

- **Tenant Escape** — in a multi-tenant MCP server, insufficient isolation lets one tenant's agent read or influence another tenant's data or session — a direct violation of the trust-boundary discipline from Stage 0.3.
- **Schema Manipulation** — tool input/output schemas are altered or under-validated, letting an agent be induced to pass malicious parameters (oversized payloads, path-traversal strings, injection payloads) that the receiving system doesn't adequately check.

3.3 Why Validating Only at Discovery Time Fails

It's tempting to think "we scanned the tool definitions before deploying them, so we're covered." This misses a specific, documented failure mode: a model that receives a clean, validated tool list can still be manipulated, at the moment it actually calls the tool, into supplying malicious parameters the original schema review never saw in action. The fix is structural, not a one-time check: gate twice. Validate the tool's definition and description when it's first registered (discovery time), and re-validate every actual call against that same schema every time the tool is invoked (invocation time). The second gate catches what the first one structurally cannot.

3.4 The MCP Gateway Pattern

Just as Stage 1.5 established that credentials should live outside the agent, this stage establishes the equivalent pattern for tool access generally: route all MCP traffic through a centralized gateway, rather than letting every developer's local configuration decide which MCP servers to trust. The gateway becomes the one place that owns policy for the whole organization, regardless of how many individual agents or developers exist.

Gateway Stage	What It Checks
1. Discovery-time schema validation	Every tool definition and description, before it ever reaches an agent's context, is checked against expected schema and scanned for hidden-instruction patterns
2. Server allowlisting	Only explicitly registered, approved MCP servers are reachable — no silent additions via local config
3. Identity and transport enforcement	Every connection requires mTLS and an authenticated workload identity (this is where Stage 2's SPIFFE work plugs directly in)
4. Policy evaluation at invocation	Every tool call is re-evaluated against policy at the moment it happens — both the caller's authorization and the call's actual parameters against the original schema
5. Tamper-evident audit logging	Every allow, deny, and policy change is logged with integrity protection, supporting the accountability principle from Stage 0.2

HANDS-ON LAB Build a Minimal MCP Gateway Policy

Your organization has three internal MCP servers (a CRM-read tool, a knowledge-base-search tool, an email-draft tool — matching the Stage 1 support agent) and is being asked to onboard a fourth, third-party MCP server from a vendor for calendar scheduling.

Tasks:

1. Write an allowlist policy (in plain pseudo-config — YAML-like is fine) naming the four approved servers and, for each, the specific tools within it that are approved for use (not blanket server-level trust).
2. For the third-party calendar server specifically, write the additional checks you would require before approving it that you would NOT require for the three internal servers (think about what changes when the publisher isn't your own organization).
3. Design the invocation-time re-validation rule for the email-draft tool specifically: what parameters does it check on every call, and what's the policy decision if the proposed email recipient list includes an address never seen for this ticket before?
4. Identify which gateway stage (from the section 3.4 table) would catch a 'rug pull' scenario where the calendar server's scheduling tool is silently modified three weeks after approval to also forward calendar invites to an external address.

Deliverable: A gateway policy document covering the allowlist, the third-party-specific additional checks, the invocation-time rule for the email tool, and the rug-pull detection answer.

Reference: EASA-02, Domain 5

KNOWLEDGE CHECK Stage 3

1. Explain the difference between tool poisoning and tool spoofing in your own words, with a one-sentence example of each that a colleague unfamiliar with MCP would understand.
2. Why is 'we scanned every tool before deploying it' not a sufficient answer to 'how do you prevent tool poisoning'? What specific gap does it leave?
3. A vendor says their MCP server doesn't need a gateway in front of it because 'it's secure by design.' What would you ask them before accepting that claim?
4. Explain, using the trust-boundary concept from Stage 0.3, why a multi-tenant MCP server is a higher-risk deployment than a single-tenant one even with identical tool definitions.

STAGE 4 • Week 5

Agent-to-Agent (A2A) Security

Where MCP governs agent-to-tool trust, A2A governs agent-to-agent trust — increasingly across organizational and vendor boundaries, with more than 150 production organizations on Google's A2A protocol as of 2026. This stage closes Phase II by extending everything you've built so far (identity, gateway patterns) to the case where the thing on the other end of the connection isn't a tool you control, but another autonomous agent you may not.

BY THE END OF THIS STAGE, YOU WILL BE ABLE TO

- Explain Agent Cards, discovery, and the role of Signed Agent Cards introduced in A2A v1.0
- Identify agent card poisoning, agent card spoofing/forgery, agent-in-the-middle, and rogue/compromised trusted agents as distinct threats
- Explain why a previously trustworthy agent is the most dangerous threat actor in an A2A ecosystem
- Design an A2A gateway with discovery validation, claims mapping, and a trust broker
- Explain the relationship between A2A's cryptographic guarantees and the cognitive-security risk that those guarantees do not address

***Prerequisite check:** Comfortable with the MCP gateway pattern (Stage 3.4) and compound identity (Stage 2.4) — most of A2A security is the same pattern applied to a peer-to-peer rather than agent-to-tool relationship.*

4.1 A2A in One Paragraph

A2A (Agent2Agent), originated by Google and now governed by the Linux Foundation under neutral, multi-vendor stewardship, lets agents discover, communicate, and collaborate as peers — including across organizational and framework boundaries, so an agent built on one company's framework can delegate a task to an agent built entirely differently at another company. Discovery works through Agent Cards: structured documents an agent publishes describing what it can do, what authentication it expects, and where to reach it. A host agent retrieves and incorporates Agent Cards into its own planning when deciding whether and how to delegate a task — which should immediately remind you of the "content becomes instructions" risk from Stage 1.1, because that's exactly what's happening here.

4.2 Why Early A2A Deployments Were Risky, and What Changed

A2A's specification, in its earlier versions, treated authentication as effectively optional, leaving credential management up to whoever implemented a given agent — the same root cause behind the MCP risk landscape in Stage 3.1. The single biggest improvement, introduced in A2A v1.0, was Signed Agent Cards: a cryptographic signature tied to the publishing domain, so a receiving agent can verify a card actually came from the domain it claims to represent, rather than trusting it at face value. This is the protocol-level fix for agent card forgery — but, importantly, signing is supported by the spec, not enforced by it, meaning unsigned deployments remain vulnerable.

4.3 Four Threats to Recognize on Sight

- **Agent Card Poisoning** — malicious instructions embedded in a Agent Card's metadata fields, which a host agent's reasoning engine incorporates into its planning context — published research demonstrates this causing unintended transmission of sensitive data (PII, payment details) in a simulated booking scenario.
- **Agent Card Spoofing / Forgery** — an attacker stands up a fake Agent Card at a domain they control, redirecting other agents to a malicious endpoint — directly mitigated, though not eliminated, by Signed Agent Cards from section 4.2.
- **Agent-in-the-Middle** — an intermediary intercepts and manipulates A2A traffic between two otherwise legitimate peers, the agent-to-agent analog of a classic network man-in-the-middle attack.
- **Rogue / Compromised Trusted Agents** — an agent that was legitimately onboarded and has behaved correctly is later compromised, and its established trust is exploited precisely because that prior good behavior makes detection less likely. This is the most operationally dangerous of the four, and worth sitting with: it means a clean track record is not evidence of current safety.

4.4 The Enterprise A2A Gateway

The same centralizing instinct from Stage 3.4 (MCP gateway) applies here, adapted for peer-to-peer trust rather than tool access:

Gateway Function	What It Does
Discovery Validation	Every inbound Agent Card is cryptographically verified against its signature before being trusted; unsigned external cards rejected by default
Claims Mapping	An external agent's claimed authority is translated into, and bounded by, your internal policy — never trusted at face value, echoing the compound-identity discipline from Stage 2.4
Trust Broker	A centralized component tracks trust relationships and current trust scores for every known external agent counterparty, rather than each internal agent making its own ad hoc trust decision
Policy Evaluation	Every delegated task is checked against policy (budget limits, data-sensitivity boundaries, allowed task types) before acceptance
Audit Logging	Full request/response capture tied to the compound identity of both sides of the interaction

4.5 What Cryptography Does Not Fix

It's worth being precise here, because this is a trap even experienced engineers fall into: signing Agent Cards and using mutual TLS secures the integrity and authenticity of a message — it proves the message came from who it claims to and wasn't tampered with in transit. It does nothing to prevent an agent from being legitimately, validly manipulated into forming a bad decision in the first place, the way prompt injection manipulates a single agent (Stage 1.1). A cryptographically perfect, fully authenticated delegation to do something harmful is still harmful. You'll see this exact point reinforced again in Stage 8 when payment protocols enter the picture — the lesson generalizes.

HANDS-ON LAB Design an A2A Trust Broker Decision

Your customer support agent (from Stages 1–3) now needs to delegate a subset of tasks — specifically, checking shipping status — to an external logistics partner's agent, reachable via A2A.

Tasks:

1. Write the discovery-validation check your gateway should perform on the logistics partner's Agent Card before any task is ever delegated to it.
2. Design a simple trust-score model for this external counterparty (you can use a 0–100 scale or a simple tiered model) and list at least three signals that should feed into it.
3. Write the policy rule that determines what trust score is required before the support agent is allowed to send a customer's shipping address and order ID to the logistics agent — and what should happen if the trust score drops mid-relationship (e.g., after a security advisory about the partner).
4. Describe, in two to three sentences, an Agent Card poisoning scenario specific to this relationship, and identify which gateway function from section 4.4 would be the primary defense.

Deliverable: A trust-broker design document: the discovery check, the trust-score model with signals, the policy rule (including the degraded-trust scenario), and the poisoning-scenario analysis.

Reference: EASA-02, Domain 6; EASA-03, Domain 13

KNOWLEDGE CHECK Stage 4

1. Explain why Signed Agent Cards reduce, but do not eliminate, A2A's spoofing risk.
2. Why is a 'rogue agent that was previously trustworthy' considered more dangerous than an obviously malicious new agent? What does this imply about how trust scores should behave over time?
3. A colleague argues that once an A2A connection uses mutual TLS and signed cards, the system is secure. What's the gap in this argument, and what concept from Stage 1 explains it?
4. Sketch, in plain language, what 'claims mapping' (section 4.4) would do differently than simply trusting whatever authority an external Agent Card claims for itself.

STAGE CAPSTONE Phase II Capstone — The Unified Trust Chain

Take the architecture sketch you produced at the end of Phase I (Stage 1 capstone) and extend it into a complete identity-to-trust design covering everything in Stages 2 through 4.

Requirements:

- A compound identity design for every agent in your system (Stage 2.4), including how it's issued and what it carries.
- A complete MCP gateway policy (Stage 3.4) for every tool your agent uses, including the two-gate validation pattern.
- If your system has any plausible agent-to-agent or cross-organization interaction, a full A2A gateway design (Stage 4.4); if it genuinely has none, write a one-paragraph justification for why, and design the gateway anyway for a hypothetical future partner integration as a forward-looking exercise.
- A single paragraph explaining, end to end, how identity flows from Stage 2 through your MCP gateway (Stage 3) and, if applicable, your A2A gateway (Stage 4) — demonstrating that you see this as one continuous chain, not three separate topics.

Graded against: Whether the design treats identity as the substrate everything else builds on (rather than treating MCP and A2A as independently solving authentication from scratch), and whether the two-gate MCP validation and

trust-broker A2A patterns are both present and correctly reasoned, not just named.

Phase III — Governance & Operations

Phase I gave you architecture. Phase II gave you the identity-to-trust chain. Phase III turns both into something an organization can actually run day to day and defend to a regulator: governance that maps to real obligations, a security operations capability that watches the system continuously, and the testing and reliability discipline that finds problems before they become incidents. This phase maps to EASA-03 and EASA-04.

STAGE 5 • Week 6

AI Governance, Risk & Compliance

Architecture without governance is a prototype that happens to be well-built. This stage gives you the regulatory and framework literacy to operate agentic systems in a real enterprise — including the urgency of the EU AI Act's August 2026 high-risk system deadline, which is likely close at hand by the time you're studying this.

BY THE END OF THIS STAGE, YOU WILL BE ABLE TO

- Explain the OWASP Top 10 for Agentic Applications (ASI01–ASI10) and map each category to a control owner
- Distinguish what identity controls can and cannot fix among the ASI categories
- Explain the relationship between ISO 42001 and the EU AI Act, including where they overlap and where real gaps remain
- Build a framework crosswalk mapping a control to multiple compliance obligations simultaneously
- Design an autonomy-level classification scheme and apply it to a real agent population

Prerequisite check: Comfortable with all of Phase II — this stage assumes you can reason about identity, MCP, and A2A controls well enough to map them onto a compliance framework.

5.1 The OWASP Top 10 for Agentic Applications (ASI01–ASI10)

This is the field's closest thing to a shared vocabulary across vendors and organizations. You should be able to recite these from memory by the end of this stage — not because memorization is the goal, but because fluent recall is what lets you participate in a real architecture review at speed.

Code	Risk	Which Stage Already Taught You This
ASI01	Agent Goal Hijack	Stage 1.1 — "content becomes instructions"
ASI02	Tool Misuse & Exploitation	Stage 3.2 — tool poisoning and related patterns
ASI03	Identity & Privilege Abuse	Stage 2 — the whole stage
ASI04	Agentic Supply Chain Vulnerabilities	Stage 3.2 — rug pulls; new ground in this stage (broader supply chain)
ASI05	Unexpected Code Execution	Stage 1.4 — runtime isolation exists specifically to contain this
ASI06	Memory & Context Poisoning	Stage 1.2 — touched on; deepened in Stage 6
ASI07	Insecure Inter-Agent Communication	Stage 4 — the whole stage
ASI08	Cascading Failures	New ground — covered fully in Stage 7
ASI09	Human-Agent Trust Exploitation	New ground — covered fully in Stage 7

Code	Risk	Which Stage Already Taught You This
ASI10	Rogue Agents	Stage 4.3 — the "previously trustworthy agent" problem

Notice that you already have working knowledge of seven of these ten categories — this stage's job is mostly to give you the shared label and to fill the three genuine gaps (ASI04's broader supply-chain scope, ASI08, and ASI09), which Stage 7 will complete.

5.2 What Identity Fixes, and What It Doesn't

It's tempting, once you've spent three weeks building identity and trust infrastructure, to believe a strong identity layer solves most agentic risk. It doesn't, and overclaiming for any single control undermines your credibility in a real review. A strong identity layer (Stage 2) is the most direct fix for ASI03, and contributes meaningfully to ASI01 and ASI07. It does nothing for a vulnerability in a third-party MCP server's own code (ASI04), a remote-code-execution bug in the runtime itself (ASI05), or a cascading failure where one bad output propagates virally through a multi-agent pipeline (ASI08). Each ASI category needs its own primary control owner — being able to say which one, and why, is a core skill of this stage.

5.3 ISO 42001 and the EU AI Act — What Actually Maps

ISO/IEC 42001 is a certifiable AI management system standard — a governance backbone covering risk management, data governance, human oversight, and continual improvement. The EU AI Act is binding law for organizations operating in or serving the EU, with a risk-based four-tier classification and a critical deadline: the Act's requirements for high-risk AI systems (Annex III) become fully applicable in August 2026. The honest relationship between the two: independent analysis finds roughly 60–70% of EU AI Act documentation requirements map directly onto existing ISO 42001 clauses, with seven core articles having direct counterparts. The gaps that remain are real — conformity assessment procedures, CE marking, the depth of Annex IV technical documentation, and statutory incident-reporting timelines. Treat ISO 42001 as the foundation, not the finish line.

A trap to avoid

Do not tell a stakeholder "we're ISO 42001 certified, so we're EU AI Act compliant." That conflation is exactly the mistake regulators and increasingly enterprise legal teams are starting to test for. The correct statement is: "ISO 42001 gives us the governance foundation; here is our documented gap analysis against the specific Act requirements that remain."

5.4 Building a Framework Crosswalk

The practical governance artifact this stage produces is a crosswalk: a table where one control, with one piece of evidence, satisfies multiple frameworks at once — instead of duplicating evidence-gathering for every regulation separately. This is the single most useful document you can hand an auditor, because it answers "can you prove this control is operating" once, in a form that's reusable across every framework that cares about it.

5.5 Autonomy Levels — Operationalizing "Least Agency"

OWASP's framing of Least Agency — autonomy is a privilege an agent earns through demonstrated reliability, not a default granted at deployment — needs a concrete scale to be enforceable rather than aspirational.

Level	Designation	Approval Requirement
L0	Advisory only	None — human executes all actions
L1	Supervised execution	Per-action human approval
L2	Bounded autonomy	Approval only above a defined risk threshold
L3	Delegated autonomy	Post-hoc audit; real-time approval only for irreversible actions
L4	Full autonomy	Periodic governance review only

HANDS-ON LAB Build a Framework Crosswalk and Classify an Agent Fleet

You're handed a list of six agents your organization runs: an internal meeting-notes summarizer, a customer support agent (from Phase II) with CRM access, an automated invoice-processing agent with payment-system access, an internal code-review agent, a marketing-content drafting agent, and an HR-policy question-answering agent.

Tasks:

1. For each of the six agents, assign an ASI category (or categories) that represents its highest residual risk, and name the primary control owner using the section 5.2 reasoning.
2. For each agent, assign an autonomy level (L0–L4) using the section 5.5 scale, with one sentence of justification each.
3. Identify which of the six agents would plausibly fall under EU AI Act Annex III high-risk classification, and explain your reasoning (note: you don't need to be a lawyer — reason from the system's purpose and consequence, e.g., does it affect employment, credit, or safety-relevant decisions).
4. Build a crosswalk table (minimum 5 rows) mapping a control area (e.g., 'agent identity & access control') to the ASI code, the relevant ISO 42001 clause area, and the relevant EU AI Act article, following the pattern in EASA-03, Domain 3.6.

Deliverable: A governance packet: the six-agent risk/autonomy table, the high-risk classification reasoning, and the crosswalk table.

Reference: EASA-03, Domains 3 and 14

KNOWLEDGE CHECK Stage 5

1. Pick any three ASI codes and explain, for each, one control that would NOT meaningfully reduce that risk even though it sounds related (e.g., 'strong identity doesn't fix ASI05 because...').
2. Explain in your own words why ISO 42001 certification is described as a 'governance maturity signal,' not a legal compliance substitute.
3. An agent currently operates at L2 (bounded autonomy). Describe a realistic event that should trigger a downgrade to L1, and explain who should be accountable for making that call.
4. Why does a framework crosswalk save real audit effort compared to addressing each regulation separately, even though the underlying controls are identical either way?

STAGE 6 • Week 7

AI SOC, Observability & Memory Security

Everything so far has been design-time: architecture, identity, governance. This stage is about what happens after launch — watching a live agentic system well enough to know the difference between "it's behaving oddly because it's a hard task" and "it's behaving oddly because something is actively wrong."

BY THE END OF THIS STAGE, YOU WILL BE ABLE TO

- Distinguish AI observability platforms (built for engineering quality) from an AI SOC capability (built for security detection and response)
- Design detection logic that correlates signals across multiple monitored surfaces rather than treating each in isolation
- Apply the memory governance lifecycle (create, classify, encrypt, retain, archive, delete, audit) to a real memory architecture
- Explain why deleting episodic memory alone does not satisfy a right-to-erasure request
- Design a kill-switch mechanism that operates at the identity layer rather than relying on agent cooperation

***Prerequisite check:** Comfortable with Stage 2 (identity) and Stage 5 (ASI categories) — this stage builds detection logic directly on both.*

6.1 Observability Is Not Security Monitoring

This distinction matters more than it might seem. Tools like LangSmith, Langfuse, and Arize Phoenix are built primarily to answer engineering questions: why did this agent loop, why did this tool call fail, why did output quality drift. They are necessary infrastructure — an AI SOC cannot exist without the trace data these platforms capture — but they are not, by themselves, a security capability. An AI SOC additionally needs security-specific detection logic (is this trace a poisoning attempt, not just a quality regression), tested response playbooks, and integration with your existing security operations team. Buying an observability platform is not the same project as building an AI SOC, even though the first is a prerequisite for the second.

6.2 Seven Surfaces, One Correlated Picture

An AI SOC monitors seven surfaces — agents, MCP, A2A, memory, runtime, guardrails, identity — and the design choice that determines whether it actually works is correlating signals across these surfaces rather than watching each in isolation. A single odd tool call is noise. The same odd tool call, immediately after an unsigned Agent Card interaction (Stage 4.3), and immediately before an unusual memory write (covered below), is a credible incident. This cross-surface correlation is, operationally, the same MAESTRO cross-layer thinking from Stage 1.3 — just running continuously against live data instead of being done once at design time.

6.3 Memory Governance Lifecycle

Agent memory is unusually hard to govern because it's written to continuously, by the agent's own judgment about what's worth remembering — unlike a conventional database where writes follow predictable application logic.

Stage	What Happens
Create	Memory entry written by the agent; source attribution captured at write time
Classify	Sensitivity/data-type classification applied synchronously, not in a later batch
Encrypt	Encryption matched to classification level; tenant-scoped keys in multi-tenant stores
Retain	Automated time-to-live tied to data type and applicable regulation
Archive	Aged-out memory moved to access-restricted storage, not casually queryable
Delete	Cryptographic erasure preferred over logical deletion for sensitive data
Audit	Every stage logged immutably, independent of the memory content itself

The single most common real-world gap in this area

Deleting an episodic memory entry ("the user asked X on this date") does nothing to the semantic or graph-memory representations already derived from it elsewhere in the system. A right-to-erasure request satisfied only at the episodic layer leaves the data subject's information persisting in a different form — meeting the letter of a narrow deletion request while violating its intent. Provenance and lineage tracking (knowing what was derived from what) is what makes erasure genuinely enforceable, not just procedurally claimed.

6.4 The Kill Switch

Every agent above the most basic autonomy level (Stage 5.5) needs a kill switch — the ability to immediately suspend its credentials and halt in-flight or queued actions. The critical design requirement: this has to operate at the identity/credential layer (revoking the agent's SPIFFE SVID and any outstanding tokens from Stage 2), not by sending the agent a polite shutdown signal and hoping it complies. A compromised or rogue agent is precisely the scenario where you cannot assume cooperation — which is the whole point of having a kill switch in the first place.

HANDS-ON LAB Design Cross-Surface Detection Logic

The invoice-processing agent from Stage 5's lab (with payment-system access) begins exhibiting an unusual pattern: three tool calls to an unfamiliar vendor-lookup endpoint in two minutes, followed by a memory write that references a vendor name not present anywhere in its recent task history.

Tasks:

1. Identify which of the seven monitored surfaces (section 6.2) each signal in this scenario belongs to.
2. Write the correlation rule that would flag this as a credible incident rather than noise — be specific about what makes the combination suspicious when neither signal alone clearly would be.
3. Apply the memory governance lifecycle (section 6.3) to determine: should this suspicious memory write be allowed to complete, quarantined, or rejected outright — and what does your answer imply about where in the write path your detection logic needs to sit?
4. Write the kill-switch trigger condition for this scenario, and specify exactly what gets revoked (referencing Stage 2's identity model) when it fires.

Deliverable: A detection design document: the surface mapping, the correlation rule, the memory-write decision with reasoning, and the kill-switch trigger and revocation scope.

Reference: EASA-04, Domains 9, 10, and 14

KNOWLEDGE CHECK Stage 6

1. Explain to a colleague who only knows observability tools (LangSmith, Langfuse) why their existing setup is necessary but not sufficient for an AI SOC.
2. Why does correlating across surfaces catch incidents that monitoring each surface independently would miss? Give a concrete example different from the one in this stage's lab.
3. A data subject submits a right-to-erasure request. Your team deletes their episodic memory entries and reports the request as fulfilled. What's missing, and why does it matter?
4. Why must a kill switch operate at the identity layer rather than by signaling the agent to stop? What assumption would you be making if you relied on the latter?

STAGE 7 • Week 8

Red Teaming, Cognitive Security & Reliability Engineering

This stage closes the remaining ASI gaps from Stage 5 (ASI08 and ASI09), gives you the offensive skill to find problems before an adversary does, and introduces Agent Reliability Engineering — treating reliability as a measurable, negotiated engineering discipline rather than an aspiration.

BY THE END OF THIS STAGE, YOU WILL BE ABLE TO

- Design and execute a red-team exercise against an agentic system, attacking across all five surfaces from the AI SOC model
- Close the loop from a red-team finding to a detection pattern and a tested response playbook (purple teaming)
- Explain the five cognitive-security threats (goal hijacking, reasoning manipulation, planning corruption, cognitive overload, agent deception) as distinct from conventional security threats
- Define agent-specific SLIs and SLOs, and design an error-budget-triggered autonomy downgrade
- Explain why reliability and security operations should share tooling and incident postmortems rather than operate as silos

***Prerequisite check:** Comfortable with all of Phase II and Stages 5–6 — red teaming this stage's lab requires you to find and chain weaknesses across identity, MCP, A2A, and detection logic you've already built.*

7.1 Red Teaming Across Five Surfaces

A conventional penetration test finds the SQL injection vulnerability in a tool your agent calls. It does not find the prompt-injection chain that gets your agent to call that tool with malicious parameters in the first place. Agentic red teaming needs both — testing conventional security flaws and agent-specific behavioral failure modes, using attack patterns specific to each surface: agents (goal-hijack via injected content), MCP (tool poisoning and schema manipulation), A2A (card spoofing, agent-in-the-middle), memory (poisoning via every available write path), and runtime (sandbox escape, resource exhaustion). Automated test suites built against the OWASP ASI categories are a good coverage floor, but the real value of human-led red teaming is chaining multiple categories together the way a real adversary would — for example, an ASI04 supply-chain foothold combined with an ASI07 inter-agent exploit to produce an ASI08 cascading failure.

7.2 Purple Teaming — Closing the Loop

A red-team report that sits in a document and is never operationalized is a wasted exercise. Purple teaming pairs every successful red-team finding with a detection pattern your AI SOC's correlation logic (Stage 6.2) can actually catch, an automated, repeatable version of the attack for ongoing regression testing, and a tested response playbook including kill-switch criteria. This converts a point-in-time finding into a durable improvement.

7.3 Cognitive Security — The Threats With No Conventional Precedent

This is where ASI08 and ASI09 finally get full treatment, and where this program's central theme from Stage 1.1 reaches its sharpest form: protecting the integrity of an agent's reasoning process itself, not just the systems around it.

- **Goal Hijacking** — an attacker manipulates the agent's interpreted objective via injected content (Stage 1.1, now with a name).
- **Reasoning Manipulation** — an attacker influences the agent's intermediate reasoning steps without changing its stated final objective, steering it off-path while appearing on-task.
- **Planning Corruption** — for agents that generate explicit plans before acting, the plan-generation step itself is corrupted — dangerous specifically because a corrupted plan can pass a superficial review that only checks the stated goal, not every step.
- **Cognitive Overload Attacks** — an attacker floods the agent's context with excessive or contradictory information specifically to degrade reasoning quality — the agentic equivalent of a denial-of-service attack, but targeting reasoning rather than availability (ASI08's mechanism, named).
- **Agent Deception** — in multi-agent systems, one agent deliberately misrepresents information to manipulate a peer's behavior — the mechanism behind agent collusion and a direct driver of ASI09 (human-agent trust exploitation, when a deceived agent then misleads a human reviewer downstream).

7.4 Agent Reliability Engineering (ARE)

Classical Site Reliability Engineering gave operations teams a quantitative vocabulary — SLIs, SLOs, error budgets — for negotiating reliability against feature velocity. Agentic systems need the same discipline, measuring failure modes classical SRE never anticipated: an agent can be perfectly available and fast while being confidently, quietly wrong.

SLI Category	What It Measures	Why It's Not a Classical-SRE Metric
Goal conformance rate	% of tasks completed without unauthorized scope expansion	Directly measures Least Agency adherence (Stage 5.5) and ASI01 exposure
Tool-call policy compliance	% of tool calls passing policy on first attempt	Surfaces over-permissioning before it becomes an incident
Memory integrity rate	% of memory writes passing provenance/classification checks	A leading indicator for poisoning risk, not just a quality metric
Human escalation accuracy	Precision and recall of human-escalation decisions	Directly measures ASI09 — both alert fatigue and dangerous under-escalation are reliability failures

Error-budget burn should trigger an automatic autonomy downgrade (back toward L1/L2 from Stage 5.5) rather than relying on a human to notice a dashboard trend — because the agents most likely to need this safeguard are, by definition, the ones with the fewest standing human checkpoints.

HANDS-ON LAB Red Team and Purple Team Your Own System

Return to the complete system you've been building across Stages 1–6 (the customer support agent and its eventual logistics/A2A extension). You will now attack it.

Tasks:

1. Design three distinct attack chains, each combining at least two ASI categories, that target your own system's specific architecture (not generic attacks — use the actual tools, agents, and trust relationships you designed in earlier labs).
2. For each attack chain, identify the cognitive-security mechanism involved (section 7.3) if one applies.
3. For each attack chain, write the corresponding purple-team output: a detection pattern your Stage 6 AI SOC design would need to catch it, and a one-paragraph response playbook.
4. Define two agent-specific SLIs (section 7.4) for your system, with target thresholds, and describe the autonomy-downgrade trigger if the error budget is exceeded.

Deliverable: A red/purple team report: three attack chains with ASI mapping and cognitive-security analysis, three corresponding detection patterns and playbooks, and two defined SLIs with downgrade triggers.

Reference: EASA-04, Domains 12, 18, and 19

KNOWLEDGE CHECK Stage 7

1. Why is a human-led red team exercise that chains multiple ASI categories more valuable than running an automated test suite against each ASI category independently?
2. Explain reasoning manipulation in a way that makes clear how it's different from goal hijacking, even though both involve an attacker influencing an agent.
3. Why should error-budget burn trigger an automatic autonomy downgrade rather than an alert that waits for a human to act on it?
4. Describe, in your own words, why reliability engineering and security operations should share an incident postmortem process rather than running as two separate functions with two separate dashboards.

STAGE CAPSTONE Phase III Capstone — Governance, Operations & Resilience Package

Extend your system design (now spanning Phases I–III) with a complete operations and governance package.

Requirements:

- The framework crosswalk and autonomy classification from Stage 5, finalized for your specific system.
- The cross-surface detection logic and memory governance design from Stage 6.
- The red/purple team report and ARE metrics from Stage 7.
- A single-page executive summary (written for a CISO or board audience, not a fellow engineer) explaining your system's current risk posture, what's mitigated, what residual risk remains, and what you'd need to do next to move it up one level on the maturity model you'll learn in Stage 9.

Graded against: Whether the executive summary is genuinely readable by a non-technical risk-owning audience (this is a distinct, gradable skill from the technical design work), and whether the red-team findings actually trace through to a specific, testable detection pattern rather than a generic recommendation.

Phase IV — Economics & Mastery

The final phase covers the part of agentic security that doesn't fit neatly into either traditional cybersecurity or traditional financial controls — agents that spend money and consume budget at machine speed — and then closes the program with a single integrated capstone that requires everything from Stages 0 through 8 at once, the way a real architect role actually demands.

STAGE 8 • Week 9

Economic Security: FinOps, Autonomous Commerce & Post-Quantum

This stage covers a category of risk most security curricula skip entirely: an agent's ability to spend, both compute budget and (increasingly) real money, faster than any human approval workflow was built to catch. It also covers a multi-year migration — post-quantum cryptography — that the identity and trust infrastructure you've spent six weeks building needs to anticipate now, even though the acute threat is still some years out.

BY THE END OF THIS STAGE, YOU WILL BE ABLE TO

- Explain budget exhaustion and cost amplification attacks, and why they are simultaneously reliability and financial incidents
- Design a unified circuit breaker triggered by either security policy violation or budget breach
- Explain how AP2 (Agent Payments Protocol) makes payment intent cryptographically verifiable, and what it does not protect against
- Apply the cognitive-security lesson from Stage 7 to a payment-agent scenario
- Explain why crypto-agility, not any single algorithm choice, is the binding constraint on post-quantum readiness

Prerequisite check: Comfortable with Stage 4 (A2A trust) and Stage 7 (cognitive security) — this stage applies both directly to financial transactions.

8.1 Cost as an Attack Surface

A misconfigured or manipulated agent in an infinite tool-call loop is not just a reliability problem (Stage 7.4) — it's a financial incident, potentially a large one, that can unfold in minutes rather than the days a traditional procurement-fraud control was designed to catch.

- **Budget Exhaustion Attacks** — an attacker deliberately drives an agent or fleet to consume its allocated budget — denying service to legitimate use, or as a direct denial-of-wallet attack. An unauthenticated A2A endpoint (Stage 4.2) is a direct enabler: it can be assigned unlimited resource-heavy tasks by any caller with no human approval gate in the path.
- **Cost Amplification Attacks** — an attacker exploits a multi-agent delegation chain so a single low-cost trigger fans out into many expensive downstream calls — the financial expression of the cascading-failure mechanism (ASI08, Stage 7.3) you already studied.

The design principle: one circuit breaker, two triggers

Build a single kill-switch and circuit-breaker mechanism (Stage 6.4) that can be triggered either by security policy violation or by budget-envelope breach, rather than building two separate emergency-stop systems owned by two different teams. In practice, runaway cost is frequently the first observable symptom of a cascading failure or a compromised agent — well before a security-specific signal fires — so treating cost anomaly detection and security anomaly detection as the same investigation workflow, not two disconnected dashboards, catches incidents earlier.

8.2 Autonomous Commerce and AP2

Google's Agent Payments Protocol (AP2), now governed via the FIDO Alliance's Payments Technical Working Group, solves a structural problem traditional payment APIs were never built for: they assume real-time human authorization at the moment of transaction, which doesn't hold when an agent acts on delegated authority, potentially asynchronously, on a task a human approved only in general terms earlier.

AP2 Concept	What It Verifies
Intent Mandate	What the user authorized in general terms ("book me a flight under \$500")
Cart Mandate	The specific transaction the agent assembled — cryptographically signed before execution
Role separation	No single role (the agent, the merchant, the processor) sees more than it needs
Verifiable settlement receipts	Deterministic, auditable proof the transaction settled as authorized

Independent research reports a meaningful fraud-rate improvement under AP2's verifiable-intent model compared to conventional API-based agent payment flows, concentrated specifically in tampering-related fraud — which makes sense, because mandate signing makes tampering cryptographically detectable rather than merely procedurally discouraged.

8.3 What AP2 Does Not Fix — Applying Stage 7 Here

This is the most important conceptual link in this stage, and it's worth being explicit about it: a verifiable-payment protocol secures the transaction, not the decision to transact. Published red-team research against AP2 confirms exactly what Stage 7's cognitive-security framework would predict — a payment protocol's cryptographic guarantees protect a mandate's integrity once it's signed, but do nothing to prevent an agent from being manipulated, via the same goal-hijacking and reasoning-manipulation mechanisms from Stage 7.3, into forming and signing a mandate it should never have formed. The cryptography is sound; the upstream risk is unchanged by it. A payment agent needs everything from Stages 1 through 7 just as much as a non-payment agent does — AP2 adds a powerful additional layer, it doesn't replace any of the earlier ones.

8.4 Post-Quantum Readiness — Why It Belongs Here, Not Just in Cryptography Class

The identity and trust infrastructure from Stage 2 (SPIFFE SVIDs), Stage 4 (signed Agent Cards), and this stage's AP2 mandates is exactly the kind of long-lived, high-value cryptographic material that "harvest-now-decrypt-later" attacks target: an adversary recording today's signed exchanges, intending to forge or decrypt them once cryptographically relevant quantum computers exist, is a realistic threat model for infrastructure you expect to remain in service for years.

Standard	Function	Practical Note
FIPS 203 (ML-KEM)	Key exchange	Already deployed in hybrid configurations in production browsers and CDNs
FIPS 204 (ML-DSA)	Digital signatures	Primary candidate for signing agent identity credentials

Standard	Function	Practical Note
		and payment mandates going forward
FIPS 205 (SLH-DSA)	Hash-based signatures (conservative fallback)	Much larger signatures; use as a crypto-agility fallback, not a primary scheme

The actual near-term action item

Post-quantum migration is a multi-year project, not a 2026 deliverable, and shouldn't compete for budget against the identity, MCP, A2A, and governance priorities from Phases II and III that address active, currently-exploited risk. What you can and should do now: ensure every new identity, MCP, A2A, and payment-mandate component built from this point forward supports algorithm rotation — crypto-agility — without a hard-coded dependency baked into firmware or an unupgradable protocol version. That single property turns the eventual PQC migration into a configuration change instead of a forklift replacement.

HANDS-ON LAB Design a Payment-Capable Agent's Economic Controls

Your invoice-processing agent (Stage 5) is being extended to autonomously approve and pay vendor invoices under \$2,000 using AP2, with anything above that routed to human approval.

Tasks:

1. Design the per-task spend cap and the FinOps budget envelope (section 8.1) as two independent, redundant controls — explain what each one catches that the other might miss.
2. Write the unified circuit-breaker trigger condition covering both a security-policy violation (e.g., an unrecognized vendor) and a budget-velocity anomaly (e.g., 10 invoices paid in 60 seconds).
3. Apply section 8.3: describe a plausible prompt-injection scenario (perhaps via a malicious invoice PDF) that could cause this agent to form a fraudulent but cryptographically valid AP2 Cart Mandate, and identify which Stage 7 cognitive-security control would be the primary defense.
4. State, in one sentence, your crypto-agility requirement for whatever signs this agent's mandates, per section 8.4.

Deliverable: An economic-controls design document: the dual-control design, the unified circuit-breaker trigger, the injection-scenario analysis with its primary defense, and the crypto-agility requirement statement.

Reference: EASA-05, Domains 15, 16, and 21

KNOWLEDGE CHECK Stage 8

1. Why should a budget-breach circuit breaker and a security-policy circuit breaker be the same mechanism rather than two separate systems?
2. Explain, to someone who thinks 'we use AP2, so our payment agent is secure,' what AP2 actually guarantees and what it leaves entirely unaddressed.
3. Why is crypto-agility described as 'the binding constraint' on post-quantum readiness, rather than the specific choice of ML-KEM vs. ML-DSA vs. SLH-DSA?
4. Describe, in your own words, why a payment-capable agent is not a fundamentally different security problem from any other agent you've designed in this program — just one where the consequences of failure are more immediately financial.

STAGE 9 • Week 10

Operating Model, Maturity & the Mastery Capstone

This final stage has two parts. First, the organizational layer that makes everything you've built operable by a real team rather than just by you: who owns what, how maturity is measured, how the work gets prioritized. Second, and the centerpiece of the entire program, the Mastery Capstone — a single integrated exercise requiring everything from Stages 0 through 8 applied to one realistic scenario, structured the way a Principal Architect's actual first ninety days on the job would be structured.

BY THE END OF THIS STAGE, YOU WILL BE ABLE TO

- Explain how the five operating-model functions (Governance Board, AI Security Office, Agent Operations, Platform Engineering, AI SOC, Red Team) divide accountability without creating silos
- Self-assess an organization against the five-level maturity model and identify a realistic next-level target
- Produce a complete, integrated security architecture, governance package, and operating plan for a novel scenario under time constraint
- Articulate, in interview-ready form, your reasoning on the core architectural tradeoffs from this entire program

Prerequisite check: All of Stages 0 through 8 — there is no new conceptual material introduced in section 9.1 and 9.2 that wasn't previewed in EASA-06; this stage is primarily synthesis and assessment.

9.1 The Operating Model in Brief

No single team owns agentic AI security end to end. The model that mirrors how mature organizations already split traditional cybersecurity, cloud platform engineering, and data governance — rather than building an isolated parallel structure — distributes accountability across five functions:

Function	Mandate
AI Governance Board	Sets policy, approves autonomy thresholds (Stage 5.5), owns the framework crosswalk (Stage 5.4), final escalation for risk-acceptance decisions
AI Security Office	Owns the architecture and policy from Stages 1-4: identity substrate, MCP/A2A gateways, governance fabric
Agent Operations Team	Day-to-day lifecycle management — registration, autonomy changes, retirement — operationalizing the Agent Registry
Platform Engineering Team	Builds and runs the runtime, gateways, and mesh infrastructure
AI SOC	Continuous monitoring and incident response (Stage 6), operates the kill switch
Red Team	Adversarial testing (Stage 7), organizationally independent to preserve objectivity

If you are an individual learner rather than part of an existing team, treat this section as the org chart you'd propose on day one of a new role — not as material you need to memorize for its own sake.

9.2 The Five-Level Maturity Model

Use this for honest self-assessment, not as a marketing artifact. Most organizations deploying agents in production today, even well-resourced ones, sit at Level 1 or the early part of Level 2 — current research is consistent that authentication is largely solved with static credentials while authorization, lifecycle management, and governance are not.

Level	Designation	Identity Pattern	Governance Pattern
1	Ad Hoc	Static API keys, shared service accounts	No central registry; no autonomy-level concept
2	Aware	Mix of static credentials and early workload-identity pilots	Registry exists but incomplete; manual periodic review
3	Managed	Ephemeral, workload-identity-issued credentials as default	Framework crosswalk operating; Governance Board meets regularly
4	Measured	Trust-score-informed dynamic authorization layered on identity	Purple-team loop closed; KPIs reported quarterly with trend data
5	Optimizing	Hybrid classical/PQC credentials; verifiable-credential external trust	Governance anticipates regulatory change; the org is a reference case

The Mastery Capstone

This is the assessment the entire program has been building toward. It is intentionally open-ended in scenario but specific in requirements — the same balance a real Principal Architect role strikes between "figure out what matters here" and "deliver something concrete by Friday."

STAGE CAPSTONE Ninety-Day Plan for a New Agentic Platform

You have just joined an organization (choose a realistic industry context — financial services, healthcare, retail, or one of your own choosing) as its first Enterprise Agentic AI Security Architect. The organization already has roughly 40 AI agents in production, deployed by individual teams with no central oversight, no consistent identity model, and no governance function. Leadership has given you ninety days to produce an assessment and a plan. Produce that plan now, compressed into this capstone.

Requirements:

- Architecture & Threat Modeling (Stages 0–1): A MAESTRO-based threat model for at least one representative agent from the fictional organization's portfolio, including a cross-layer threat and a runtime isolation decision.
- Identity, MCP & A2A Design (Stages 2–4): A target-state identity architecture (compound identity model), an MCP gateway design, and an A2A trust-broker design — written as a migration plan from the organization's current ad hoc state, not just a greenfield design.
- Governance & Compliance (Stage 5): A framework crosswalk and an autonomy-level classification scheme applied to a representative slice of the 40-agent portfolio, plus an explicit call on whether any agents plausibly fall under EU AI Act high-risk classification.
- Operations & Resilience (Stages 6–7): An AI SOC design (the seven monitored surfaces, with at least one cross-surface correlation rule), a memory governance lifecycle application, and a red-team plan for the first 90 days with at least two attack chains.
- Economic Security (Stage 8): If the organization's portfolio plausibly includes any agent with cost or payment exposure, a FinOps/circuit-breaker design for it; if not, a one-paragraph justification and a forward-looking design for the most likely next payment-capable use case.
- Operating Model & Maturity (Stage 9): A proposed operating-model org structure (using the five functions from section 9.1, adapted to the organization's actual size), a maturity self-assessment (the organization starts at Level 1), and a realistic 12-month target level with the three highest-priority workstreams to get there.
- Executive Summary: A two-page summary written for the CEO and board — not a fellow architect — covering current risk posture, the plan, and what resources (headcount, budget, timeline) you're requesting.

Graded against: (1) Whether the plan is genuinely sequenced and prioritized — does it identify the identity substrate as the highest-leverage early investment, consistent with the central lesson of this entire program, rather than treating all seven sections as equally urgent in parallel; (2) technical correctness within each section, evaluated against the standards in Stages 0–8; (3) whether the executive summary is actually readable and persuasive to a non-technical, risk-owning audience; (4) realism — does the 90-day and 12-month plan reflect what a real organization with finite budget and finite people could actually execute, or does it read as an unprioritized wish list.

Recommended Time Allocation for the Capstone

Section	Suggested Time
Research & scenario definition (choosing your industry context, sketching the	0.5 day

Section	Suggested Time
fictional agent portfolio)	
Architecture & threat modeling	1 day
Identity, MCP & A2A design	1.5 days
Governance & compliance	0.5 day
Operations & resilience	1 day
Economic security	0.5 day
Operating model, maturity & executive summary	1 day

After the Capstone: Where to Go Next

Completing this curriculum gets a learner to working competence — able to do the four things named in this document's opening (threat-model, design, govern, operate) without supervision on a bounded, realistic scenario. It does not, by itself, make someone a Principal Architect with the scar tissue of having lived through real incidents, real vendor negotiations, and real regulatory audits. That gap closes through practice, not further reading. Three concrete next steps:

1. Pursue a credential that validates this knowledge externally — CAISP (Certified AI Security Professional) is the broadest single AI security certification as of 2026 and maps closely to this curriculum's balance of offense, defense, and governance; the Principal Architect Interview Guide in EASA-06 is a useful self-test before attempting it.
2. Apply this curriculum's pattern — threat model, identity design, gateway design, governance crosswalk, operational plan — to a real system at your own organization, even a small one, rather than only the fictional scenarios in this document. The fictional scenarios were deliberately realistic but deliberately bounded; a real system will surprise you in ways a curriculum cannot.
3. Track the standards landscape directly going forward, not just this curriculum's snapshot of it. NIST's AI Agent Standards Initiative, the IETF WIMSE working group, the OWASP MCP Top 10 (in beta as of this writing), and the FIDO Alliance's AP2 post-quantum profile were all explicitly still in motion at the time this program was written — re-verify the current state of each before making a significant architecture investment based on them.

Six volumes and ten stages converge on one sentence, repeated because it is the single highest-leverage thing a learner of this program can act on: build the identity substrate first. Every other domain in this curriculum — MCP gateways, A2A trust, governance crosswalks, FinOps circuit breakers, even post-quantum migration — gets simpler, cheaper, and more defensible once every agent, tool, and peer in the ecosystem carries a verifiable, ephemeral, centrally governed identity. You now know why. Go build it.